KASTRA POS

Admin & Installation Guide

KASTRA Technologies

18 August 2026

Contents

KASTRA POS — Admin & Installation Guide **2**
Contents . . . 2
1. What you are deploying . . . 2
2. Requirements . . . 3
3. Install on a server . . . 3
4. Configuration . . . 5
5. Running it in production . . . 6
6. Nginx and SSL . . . 6
7. Deploying to Render . . . 7
8. The desktop app . . . 7
9. The mobile apps . . . 8
10. First-run setup . . . 8
11. eTIMS and payment credentials . . . 8
12. Backups . . . 8
13. Upgrades and migrations . . . 9
14. Monitoring and health . . . 9
15. Security checklist . . . 9
16. Troubleshooting . . . 10

KASTRA POS — Admin & Installation Guide

This guide is for whoever installs and looks after the server. If you are trying to take a sale or close a shift, you want the **User Guide** instead.

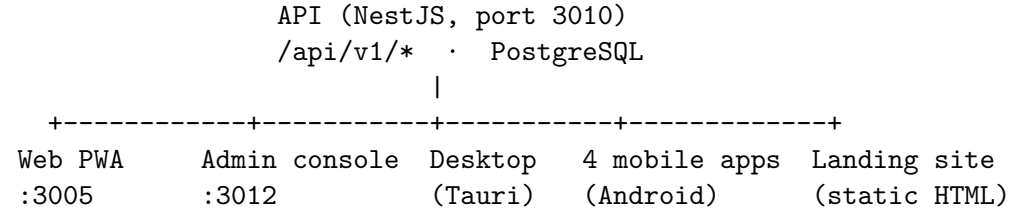
Contents

1. What you are deploying
2. Requirements
3. Install on a server
4. Configuration
5. Running it in production
6. Nginx and SSL
7. Deploying to Render
8. The desktop app
9. The mobile apps
10. First-run setup
11. eTIMS and payment credentials
12. Backups
13. Upgrades and migrations
14. Monitoring and health
15. Security checklist
16. Troubleshooting

---

1. What you are deploying

One backend serves everything:



Piece	What it is	Must you deploy it?
API	NestJS backend, the only thing that talks to the database	Yes
Web	The POS itself, a React PWA	Yes
Admin	Platform console for managing tenants	Only if you run KASTRA as a service
Landing	Static marketing site	Optional
Desktop	Tauri shell around the web app, adds printer/drawer/LAN sync	Optional
Mobile	Four Android apps	Optional

2. Requirements

Server

	Minimum	Comfortable
CPU	2 cores	4 cores
RAM	4 GB	8 GB
Disk	20 GB SSD	50 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS

Software

- **Node.js 20+** (the repo pins 20.11.1 in `.node-version`; CI builds on 22)
- **PostgreSQL 16+**
- **npm** — this project uses npm workspaces. It is **not** a pnpm project
- **Redis** — optional. Without it, background jobs run synchronously, which is fine for a single shop and not fine at volume
- **Nginx** — to put it on port 443
- **Rust toolchain** — only if you build the desktop app

Client devices — any modern browser. The web app installs as a PWA.

3. Install on a server

3.1 System packages

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git build-essential nginx

# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version    # expect v20.x
```

3.2 PostgreSQL

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
```

Create the database and user. Choose your own password and keep it out of any file you commit:

```
sudo -u postgres psql << 'EOF'
CREATE DATABASE kastra_pos;
CREATE USER kastra WITH PASSWORD 'CHANGE_THIS';
GRANT ALL PRIVILEGES ON DATABASE kastra_pos TO kastra;
ALTER DATABASE kastra_pos OWNER TO kastra;
\c kastra_pos
GRANT ALL ON SCHEMA public TO kastra;
ALTER USER kastra CREATEDB;
EOF
```

3.3 Redis (optional)

```
sudo apt install -y redis-server
sudo systemctl enable --now redis-server
redis-cli ping        # expect PONG
```

3.4 The application

```
sudo mkdir -p /opt/kastra-pos && sudo chown $USER:$USER /opt/kastra-pos
cd /opt/kastra-pos
git clone <your-repository-url> .

npm install
npm run install-all                                    # api + web + admin
npx prisma generate --schema prisma/schema.prisma      # from the repo ROOT
```

> **Do not use `npm run setup`.** It ends with `cd apps/api && npx prisma generate` without a `--schema` flag, and the schema lives at `prisma/schema.prisma` in the repo root — so it fails with *"Could not find a schema"*. Use the three commands above.

Every Prisma command needs an explicit `--schema`. There is no `apps/api/prisma/` directory.

3.5 Migrate and seed

```
npx prisma migrate deploy --schema prisma/schema.prisma

# Optional demo data for evaluating the system - NOT for a real deployment:
cd apps/api && npx ts-node src/seed.ts
```

The seed creates a demo business with sample staff logins. **Never run it on a production system you intend to trade on**, and if you do run it to evaluate, delete those users before going live.

3.6 Build

```
cd /opt/kastra-pos
npm run build        # api + web + admin
```

4. Configuration

```
cp apps/api/.env.example apps/api/.env
```

Required

```
DATABASE_URL="postgresql://kastra:CHANGE_THIS@localhost:5432/kastra_pos"
JWT_SECRET="<openssl rand -base64 32>"
ENCRYPTION_KEY="<openssl rand -hex 32>"
FRONTEND_URL="https://pos.your-domain.com"
ADMIN_URL="https://admin.your-domain.com"
```

Generate the secrets:

```
openssl rand -base64 32     # JWT_SECRET
openssl rand -hex 32        # ENCRYPTION_KEY - must be exactly 64 hex characters
```

`ENCRYPTION_KEY` encrypts M-Pesa and eTIMS credentials at rest with AES-256-CBC. **If you lose it, those stored credentials cannot be decrypted** and must be re-entered. Back it up somewhere safe and separate from the database.

`FRONTEND_URL` is not decorative. It is used server-side to build the links in emails, SMS receipts, delivery tracking and supplier RFQ invitations. Leave it wrong and your customers get `localhost` links.

Optional — each blank value simply disables that feature

Variable	Enables
`REDIS_HOST`	Background job queues with retries
`MPESA_*`	M-Pesa STK push and QR
`AT_USERNAME`, `AT_API_KEY`	Africa's Talking SMS and WhatsApp
`RESEND_API_KEY`, `EMAIL_FROM`	Transactional email
`PAYSTACK_SECRET_KEY`, `PAYSTACK_*_PLAN`	Card payments and subscriptions
`SENTRY_DSN`	Error tracking
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Google sign-in
`ETIMS_ENV`	`test` (default) or `production`
`SUPER_ADMIN_EMAIL`, `SUPER_ADMIN_PASSWORD`, `SUPER_ADMIN_SECRET`	The admin console login
`EXTRA_ORIGINS`	Additional allowed CORS origins, comma-separated

`.env` is never committed to git.

5. Running it in production

Use a process manager. PM2 is the simplest:

```
sudo npm install -g pm2

cd /opt/kastra-pos/apps/api
pm2 start dist/main.js --name kastra-api
pm2 save
pm2 startup        # then run the command it prints
```

Useful:

```
pm2 status
pm2 logs kastra-api
pm2 restart kastra-api
```

The web and admin apps build to static files (`apps/web/dist`, `apps/admin/dist`) — Nginx serves those directly, no process needed.

6. Nginx and SSL

The web app calls the API at the relative path `/api/v1`, so the cleanest setup serves both from one origin and proxies `/api` through.

```
server {
    listen 80;
    server_name pos.your-domain.com;

    root /opt/kastra-pos/apps/web/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api {
        proxy_pass http://localhost:3010;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

The admin console is a second server block pointing at `apps/admin/dist`, on its own hostname.

```
sudo ln -s /etc/nginx/sites-available/kastra /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d pos.your-domain.com -d admin.your-domain.com
```

Certbot configures renewal automatically.

Set `VITE_API_URL` only if the API is on a different origin than the web app. Left unset, the app uses the relative `/api/v1` and adapts to whatever domain serves it — which is what you want with the config above.

7. Deploying to Render

`render.yaml` in the repo root is a working blueprint.

- **Build:** `npm ci && bash scripts/render-build.sh`
- **Start:** `prisma migrate deploy` then `node dist/main`
- **Health check:** `/api/v1/health`
- **Region:** Frankfurt is the closest to Kenya

Set `DATABASE_URL` and `DIRECT_URL` to your managed PostgreSQL.

> **Supabase gotcha:** `DIRECT_URL` must use the **session** pooler on port 5432, not the transaction pooler on 6543. Migrations hang on the transaction pooler.
>
> **Render free tier** sleeps after 15 minutes idle. `KEEP_ALIVE=true` self-pings every 14 minutes to reduce cold starts, but a shop that trades all day should be on a paid instance.
>
> **Render's free PostgreSQL is deleted after 90 days.** Use Supabase, Neon, or a paid database. Do not put a real business on it.

8. The desktop app

The desktop app is a Tauri shell that **bundles the built web app**, adding native capability the browser cannot reach: direct printer sockets, cash-drawer kick, LAN peer sync, an offline eTIMS queue in SQLite, and machine-bound licensing.

```
cd apps/web && npm run build      # MUST be first - Tauri bundles apps/web/dist
cd ../desktop && npm run build
```

Installers appear in `apps/desktop/src-tauri/target/release/bundle/`.

Network topologies — chosen at first run:

- **Standalone** — one terminal, own local queue
- **Server** — hosts the API for the site, answers LAN discovery broadcasts, pushes sync events to clients
- **Client** — finds the server on the LAN and syncs to it

Client terminals also pick a **device role**: full POS, cashier-only, kiosk, kitchen display, or manager.

LAN discovery uses UDP port **47474**. Allow it between terminals.

> **Known issue:** `apps/desktop/src-tauri/tauri.conf.json` sets `devUrl` to `http://localhost:3001`, but the web dev server runs on **3005**. This affects `npm run dev` only, not release builds.

9. The mobile apps

Four Android apps, built by the **Mobile Apps Release** GitHub workflow (or a `mobile-v*` tag). Each is outside the npm workspace and installs from its own directory.

Pointing them at your server: each app's login screen accepts a server URL. To ship a different default, set `EXPO_PUBLIC_API_URL` at build time.

> **Plain-HTTP LAN servers need cleartext traffic enabled on Android.** If the apps can reach the server in a browser but not in the app, this is why.

Versioning: `scripts/bump-mobile-version.mjs <version> [app]` sets `version` in `app.json`. `versionCode` and iOS `buildNumber` are owned by EAS. `version` is also the OTA runtime key — bumping it cuts existing installs off from updates published against the old version, so bump it when native code changed and a rebuild is going out anyway, not for a JavaScript fix you intend to ship with `eas update`.

10. First-run setup

1. Open the web app and complete the **setup wizard**
2. Create the business — name, KRA PIN, tax type (VAT or TOT)
3. Choose the **business type**; it determines the whole POS layout
4. Create your first **store**
5. Add **staff** with roles and PINs
6. Load **items** (CSV import for anything more than a handful)
7. Configure **receipt** and **printers**
8. Enter **eTIMS** credentials in test mode and submit a test sale
9. Enter **payment** credentials
10. Take a real sale end to end before you open the doors

11. eTIMS and payment credentials

Entered in the app under Settings, encrypted at rest with `ENCRYPTION_KEY`. They are never in the codebase and never in `.env`.

eTIMS — start with `ETIMS_ENV=test`. Submit test sales, confirm they land, then switch to `production` once KRA approves the device. Sales queue locally when KRA is unreachable and drain when it returns.

M-Pesa — Daraja credentials. `MPESA_ENV` starts at `sandbox`.

Paystack — needed for both card payments and subscription billing. The webhook signature is verified against the raw request body.

12. Backups

Nothing here is optional for a business that trades on this system.

```
# Daily dump
pg_dump -U kastra kastra_pos | gzip > /backups/kastra_$(date +%F).sql.gz
```

Add it to cron, keep copies off the machine, and **test a restore** — an untested backup is a hope, not a backup.

Also back up: - `apps/api/.env` — specifically `ENCRYPTION_KEY`, without which stored eTIMS and M-Pesa credentials are unrecoverable - Any uploaded files

The desktop app has its own local backup/restore for standalone terminals.

13. Upgrades and migrations

```
cd /opt/kastra-pos
pg_dump -U kastra kastra_pos | gzip > /backups/pre-upgrade.sql.gz    # always

git pull
npm install && npm run install-all
npx prisma migrate deploy --schema prisma/schema.prisma
npm run build
pm2 restart kastra-api
```

Migration history is consolidated onto a base migration (`prisma/migrations/20260314000000_init`). Validate before deploying:

```
npx prisma validate --schema prisma/schema.prisma
```

14. Monitoring and health

Endpoint	Tells you
`GET /api/v1/health`	The service is up
`GET /api/v1/health/config`	Which environment variables are **set** — booleans only, never values

`/health/config` is safe to expose and is the fastest way to diagnose a deployment where a feature silently does nothing: a blank credential disables its feature without erroring.

The admin console shows live queue depths, eTIMS failure rate, and a platform alert feed, deriving an overall status of Operational / Degraded / Incident.

Set `SENTRY_DSN` for error tracking.

15. Security checklist

- ☐ `JWT_SECRET` and `ENCRYPTION_KEY` are freshly generated, not copied from a guide
- ☐ `ENCRYPTION_KEY` is backed up somewhere separate from the database
- ☐ `.env` is not in git and is readable only by the service user
- ☐ Demo/seed users deleted before going live
- ☐ HTTPS everywhere; certbot renewal working
- ☐ `FRONTEND_URL` and `ADMIN_URL` set correctly — they are the CORS allowlist
- ☐ PostgreSQL not exposed to the internet
- ☐ Firewall allows only 22, 80, 443 (plus UDP 47474 internally for LAN sync)
- ☐ Two-factor enabled for owner accounts
- ☐ Backups running **and a restore tested**
- ☐ Strong `SUPER_ADMIN_PASSWORD` if the admin console is deployed

```
sudo ufw allow 22,80,443/tcp
sudo ufw enable
```

16. Troubleshooting

`Could not find a schema` on any Prisma command Pass `--schema`. From the repo root: `--schema prisma/schema.prisma`. From `apps/api`: `--schema ../../prisma/schema.prisma`.

`Cannot find module '@pos/shared'` The shared package has not been built. `npm run build --workspace=packages/shared`. The API's `prebuild` normally does this.

API starts, web shows a blank page or 404s on the API Check the Nginx `/api` proxy, and that `VITE_API_URL` is *unset* when both are on the same origin.

CORS errors in the browser console The origin is not on the allowlist. Set `FRONTEND_URL` / `ADMIN_URL` correctly, or add it to `EXTRA_ORIGINS`. Unlisted origins are denied silently.

A feature does nothing and logs no error Its credential is blank. Check `/api/v1/health/config`.

Migrations hang on Supabase `DIRECT_URL` is on the transaction pooler (6543). Move it to the session pooler (5432).

Vite fails with `ENOSPC` during a build Raise the inotify watcher limit:

```
sudo sysctl -w fs.inotify.max_user_watches=524288
echo 'fs.inotify.max_user_watches=524288' | sudo tee -a /etc/sysctl.conf
```

Jobs are not retrying `REDIS_HOST` is unset, so jobs run synchronously with no retry. Fine for one shop, not at volume.

Desktop app shows an old version of the POS `apps/web` was not rebuilt before the desktop build. Tauri bundles `apps/web/dist`.

Mobile apps cannot reach a LAN server Enable cleartext traffic for plain HTTP, and confirm the device is on the same network.

---

*Day-to-day operation is covered in the **User Guide**. Platform-level tenant administration is in the **Super Admin Guide**, inside the admin console.*